

OSSP Skill Week 2

Title

To Analyze Process Abstraction, Execute fork(), Understand the exec() Family, Analyze Parent-Child Relationships, Inspect the Process Tree, Practice System Call Tracing.

Aim

To understand how a new process is created with fork(), how its image is replaced with exec(), and how the parent process waits for the child to finish.

Task 1: Understanding fork() and exec() Using Manual Commands

Procedure

1. Read the manual pages using `man 2 fork` and `man 3 exec` to study the syntax, return values and behaviour of each call.
2. Display the running processes with their parent IDs using `ps -ef`.
3. Display the process hierarchy using `pstree -p` to see parent-child relationships.
4. Trace the system calls made by a command using `strace ls` and identify the `execve()` call.

Output

```
EXEC(3)                                Library Functions Manual                                EXEC(3)

NAME
    execl, execl_e, execlp, execv, execvp, execvp - execute a file

LIBRARY
    Standard C Library (libc, -lc)

SYNOPSIS
#include <unistd.h>

extern char **environ;

int
execl(const char *path, const char *arg0, ..., /*, (char *)0, */);

int
execl_e(const char *path, const char *arg0, ..., /* (char *)0 char *const envp[] */);

int
execlp(const char *file, const char *arg0, ..., /*, (char *)0, */);

int
execv(const char *path, char *const argv[]);

int
execvp(const char *file, char *const argv[]);

:
```

```
FORK(2)                                System Calls Manual                                FORK(2)

NAME
    fork - create a new process

SYNOPSIS
#include <unistd.h>

pid_t
fork(void);

DESCRIPTION
fork() causes creation of a new process. The new process (child process) is an exact copy of the
calling process (parent process) except for the following:

    • The child process has a unique process ID.

    • The child process has a different parent process ID (i.e., the process ID of the parent
      process).

    • The child process has its own copy of the parent's descriptors. These descriptors
      reference the same underlying objects, so that, for instance, file pointers in file
      objects are shared between the child and the parent, so that an lseek(2) on a descriptor
      in the child process can affect a subsequent read or write by the parent. This
      descriptor copying is also used by the shell to establish standard input and output for
      newly created processes as well as to set up pipes.

:
```

```
r --blink-settings=dnsPrefetchingEnabled=false --lang=en-GB --num-raster-threads=4 --enable-zero-copy --enable-gpu-memory-buffer-compositor-resources
--enable-main-frame-before-activation --renderer-client-id=68 --launch-time-ticks=96586828782 --shared-files --field-trial-handle=1718379636,r,8110510
165520410072,18115938180342013928,262144 --enable-features=DocumentPolicyIncludeJSCallStacksInCrashReports,IsolatesPriorityUseProcessPriority,PdfUseSh
owSaveFilePicker,ScreenCaptureKitPickerScreen,ScreenCaptureKitStreamPickerSonoma --disable-features=DropInputEventsWhilePaintHolding,LocalNetworkAcces
sChecks,ScreenAIOCREnabled,SpareRendererForSitePerProcess,TimeoutHangingVideoCaptureStarts,TraceSiteInstanceGetProcessCreation --variations-seed-versi
on --pseudonymization-salt-handle=1935764596,r,13510396671176754793,5046285104156873187,4 --trace-process-track-uuid=3190709050030717226 --seatbelt-cl
ient=45
501 45070      1  0  9:36AM ??      0:00.12 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45071      1  0  9:36AM ??      0:00.10 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45072      1  0  9:36AM ??      0:00.10 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45073      1  0  9:36AM ??      0:00.08 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45075      1  0  9:36AM ??      0:00.11 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45153      1  0  9:37AM ??      0:00.07 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45154      1  0  9:37AM ??      0:00.08 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45155      1  0  9:37AM ??      0:00.13 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A
/Support/mdworker_shared -s mdworker -c MDSImporterWorker -m com.apple.mdworker.shared
501 45228     1351  0  9:37AM ??      0:00.00 (disclaimer)
0 45229 45228  0  9:37AM ??      0:00.03 (ps)
501 44184     29073  0  9:35AM ttys001 0:00.23 /bin/zsh -il
0 45237 44184  0  9:37AM ttys001 0:00.01 ps -ef
o hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The manual pages described the return values of `fork()` and `exec()`. The process tree showed every process linked to a parent, and `strace` showed that running a command internally uses `execve()`.

Result

The working of the `fork()` and `exec()` system calls was understood using manual commands.

Task 2: Creating a Process and Executing User Commands

Aim

To write a C program that reads a Linux command from the user, creates a child process using `fork()` and executes the command in the child using `execlp()`, while the parent waits.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    char command[100];

    printf("Enter a Linux command: ");
    if (scanf("%99s", command) != 1) {
        fprintf(stderr, "Failed to read command.\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Child process: replace its image with the command */
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        execlp(command, command, (char *)NULL);

        /* Reached only if execlp() fails */
        perror("Command execution failed");
        exit(1);
    } else {
        /* Parent process: wait for the child to finish */
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child finished execution.\n");
    }

    return 0;
}
```

Output

```
0 45237 44184 0 9:37AM ttys001 0:00.01 ps -ef
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc fork.exec.c -o fork.exec
● hrushika@Hrushikas-MacBook-Air pjsdj % ./fork.exec
Enter a Linux command: ls
Parent Process
Parent PID: 45425
Child Process
Child PID: 45432
fork.exec      w10q1.c      w10q2      w11q1      w11q2      w9q1      w9q2
fork.exec.c    w10q11.sh    w10q2.c    w11q1.c    w11q2.c    w9q1.c    w9q2.c
Child finished execution.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The parent and child processes ran with different PIDs. The child replaced its image with the entered command, which printed the current directory, and the parent resumed only after the child finished.